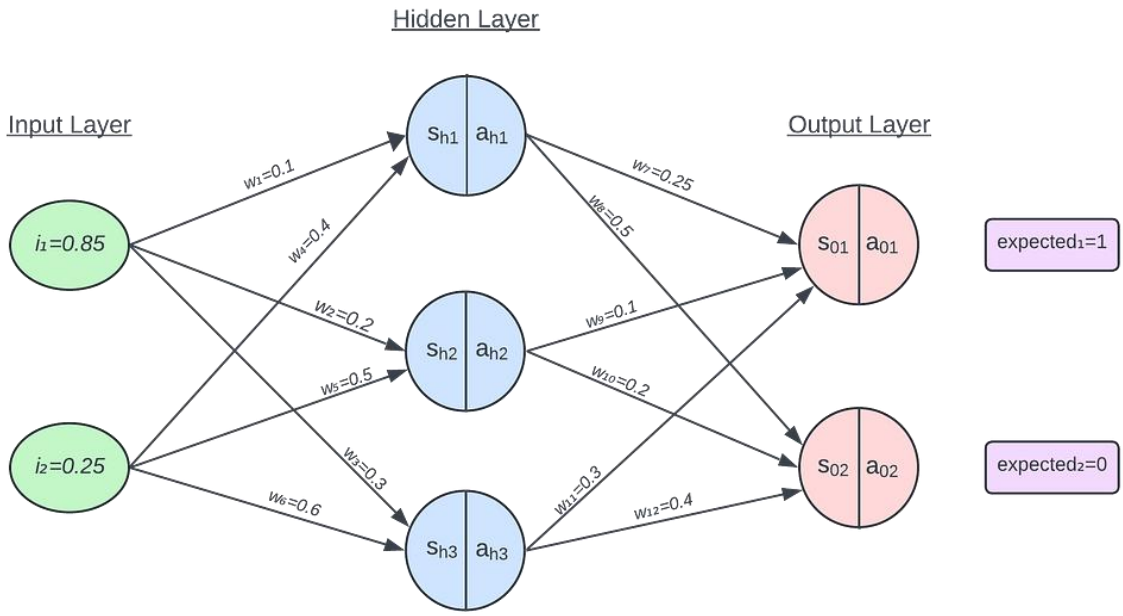
Neural Network Basics

The example we will be using is:

Zoom image will be displayed



Typically, the **input layer** (green) are the **input variables** from a dataset, and the **output layer** (red) is the neural network's **prediction value**. Within the hidden and output layers, a **weighted sum** (denoted by *s*) is taken for the each node, followed by the application of an **activation function** (denoted by *a*) which normalises the value according to a desired activation function.

The process of feeding data through a network from input to output is called **forward propagation.** The process of observing the error rate of the forward propagation and feeding the error backward to fine-tune the weights of the neural network is called **back propagation**. We forward propagate before we back propagate.

Forward Propagation

Note: I am using the **sigmoid** function as the activation function for this example (activation functions are used as mappings for inputs to be within certain ranges — in the case of sigmoid, the range is (0, 1)).

Zoom image will be displayed

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Hidden Layer

Hidden Layer 1:

Zoom image will be displayed

$$S_{h1} = (w_1 \times i_1) + (w_4 \times i_2)$$
$$= (0.1 \times 0.85) + (0.4 \times 0.25)$$
$$= 0.185$$

Zoom image will be displayed

$$a_{h1} = \frac{1}{1 + e^{-0.185}} = \boxed{0.5461}$$

Hidden Layer 2:

Zoom image will be displayed

$$S_{h2} = (w_2 \times i_1) + (w_5 \times i_2)$$
$$= (0.2 \times 0.85) + (0.5 \times 0.25) = 0.295$$

Zoom image will be displayed

$$a_{h2} = \frac{1}{1 + e^{-0.295}} = \boxed{0.5732}$$

Hidden Layer 3:

Zoom image will be displayed

$$S_{h3} = (w_3 \times i_1) + (w_6 \times i_2)$$
$$= (0.3 \times 0.85) + (0.6 \times 0.25) = 0.405$$

Zoom image will be displayed

$$a_{h3} = \frac{1}{1 + e^{-0.405}} = \boxed{0.5998}$$

Output Layer

Output Layer 1:

Zoom image will be displayed

$$S_{01} = (w_7 \times a_{h1}) + (w_9 \times a_{h2}) + (w_{11} \times a_{h3})$$
$$= (0.25 \times 0.5461) + (0.1 \times 0.5732) + (0.3 \times 0.5998) = 0.3738$$

Zoom image will be displayed

$$a_{01} = \frac{1}{1 + e^{-0.3738}} = \boxed{0.5924}$$

Output Layer 2:

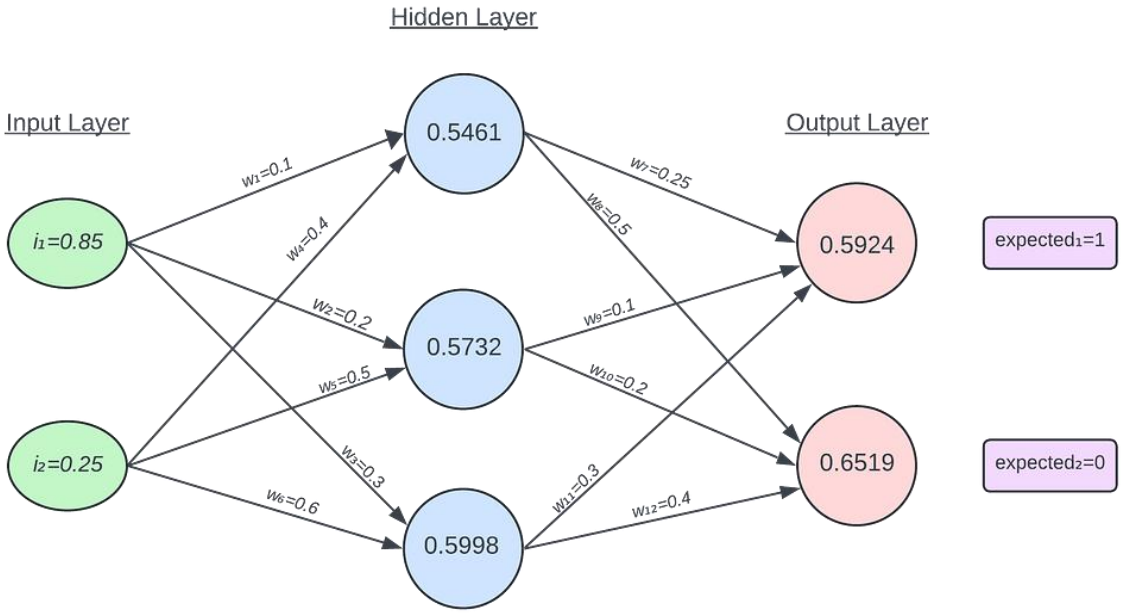
Zoom image will be displayed

$$S_{02} = (w_8 \times a_{h1}) + (w_{10} \times a_{h2}) + (w_{12} \times a_{h3})$$
$$= (0.5 \times 0.5461) + (0.2 \times 0.5732) + (0.4 \times 0.5998) = 0.6276$$

Zoom image will be displayed

$$a_{02} = \frac{1}{1 + e^{-0.6276}} = \boxed{0.6519}$$

Zoom image will be displayed



Mean Squared Error (MSE) Calculation

The mean squared error is a measure of the difference between the expected and actual outputs. We are looking for a low MSE score, which indicates a better fit of the model to the data. We will be using **gradient descent** as a way to decrease this value.

Zoom image will be displayed

$$\text{mean squared error} = \frac{1}{2} \times \sum (expected - actual)^2$$

$$= \frac{1}{2}[(1 - 0.5924)^2 + (0 - 0.6519)^2]$$

$$= \boxed{0.2956}$$

Back Propagation

Now that the predicted value is calculated, the neural network needs to adjust its weights based on the prediction error. This is done through back propagation.

For this example, consider a learning rate of 0.1

Zoom image will be displayed

$$\alpha = 0.1$$

The general mathematical idea behind back-propagation is to apply the chain rule to find the change in the error function over the change in a weight. Consider weight 7 for this example:

Zoom image will be displayed

$$\frac{\partial \text{expected}_1}{\partial w_7} = \frac{\partial \text{expected}_1}{a_{01}} \times \frac{\partial a_{01}}{\partial S_{01}} \times \frac{\partial S_{01}}{\partial w_7}$$

All three partial equations can be derived from our work.

Firstly,

Zoom image will be displayed

$$\text{expected} = \frac{1}{2} \sum_{i=1}^{n} (a_{0i} - \text{expected}_i)^2$$

$$\therefore \frac{\partial expected_1}{a_{01}} = 2 \times \frac{1}{2} \times a_{01} - \text{expected}_1$$

$$= a_{01} - \text{expected}_1$$

Secondly,

Zoom image will be displayed

$$\frac{\partial a_{01}}{\partial s_{01}}$$

$$= \frac{\partial}{\partial s_{01}} \sigma(S_{01})$$

$$= a_{01} \times (1 - a_{01})$$

And lastly,

Zoom image will be displayed

$$\frac{\partial S_{01}}{\partial w_7} = \frac{\partial (w_7 \times a_{01} + w_8 \times a_{02})}{\partial w_7}$$
$$= a_{h1}$$

Hence, putting all three terms together,

Zoom image will be displayed

$$\frac{\partial \text{expected}_1}{\partial w_7} = (a_{01} - \text{expected}_1) \times a_{01} \times (1 - a_{01}) \times a_{h1}$$

This formula can be done for all weights connecting the hidden layer to the output layer.

note: often authors may write the equation using a delta: $\delta_{o1} = (a_{o1} - expected_1) \times a_{o1} \times (1 - a_{o1})$, so the equation can be written as $\partial E_{o1} / \partial w_7 = \delta_{o1} \times a_{h1}$

Now we have the gradient of the error function.

We want to apply gradient descent to get a new value of weight w_7. The new w_7 (we can symbolise this w_7') can be obtained by subtracting the learning rate multiplied by the gradient from w_7.

Zoom image will be displayed

$$w_7' = w_7 - \frac{\partial E_{01}}{\partial w_7}$$
$$= w_7 - \alpha \times \delta_{01} \times a_{h1}$$

So in general, for an output neuron:

Zoom image will be displayed

$$w_{new} = w - \alpha \times \delta_0 \times a_h,$$
$$\text{where } \delta_o = (a_o - expected) \times a_o \times (1 - a_0)$$

Output Layer

Now, applying real numbers from the example to find new values of w_7 through w_{12}

Output Layer 1:

Zoom image will be displayed

$$\begin{aligned}\delta_{01} &= (a_{01} - \text{expected}) \times a_{01} \times (1 - a_{01}) \\ &= (0.5924 - 1) \times 0.5924 \times (1 - 0.5924) \\ &= -0.0984\end{aligned}$$

Zoom image will be displayed

$$\begin{aligned}w_7' &= w_7 - \alpha \times \delta_{0_1} \times a_{h_1} \\ &= 0.25 - 0.3 \times -0.0984 \times 0.5461 \\ &= \boxed{0.2661}\end{aligned}$$

Zoom image will be displayed

$$\begin{aligned}w_9' &= 0.1 - 0.3 \times -0.0984 \times 0.5732 \\ &= \boxed{0.1169}\end{aligned}$$

Zoom image will be displayed

$$\begin{aligned}w_{11}' &= 0.3 - 0.3 \times -0.0984 \times 0.5998 \\ &= \boxed{0.3177}\end{aligned}$$

Output Layer 2:

Zoom image will be displayed

$$\begin{aligned}\delta_{02} &= (a_{02} - \text{expected}) \times a_{02} \times (1 - a_{02}) \\ &= (0.6519 - 0) \times 0.6519 \times (1 - 0.6519) \\ &= 0.1479\end{aligned}$$

Zoom image will be displayed

$$w_8' = 0.5 - 0.3 \times 0.1479 \times 0.6517$$
$$= \boxed{0.4710}$$

Zoom image will be displayed

$$w_{10}' = 0.2 - 0.3 \times 0.1479 \times 0.5732$$
$$= \boxed{0.1746}$$

Zoom image will be displayed

$$w_{12}' = 0.4 - 0.3 \times 0.1479 \times 0.5998$$
$$= \boxed{0.3734}$$

The Hidden Layer (Derivation)

Finding a way to optimise the weights for the hidden layer has a much much larger derivation — none of this section is relevant to the calculations, so feel free to skip this part if need be.

Consider updating the weights for w_1 — in principle, updating any weight will have the same style of formula in terms of revolving around partial differentiations.

Zoom image will be displayed

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial a_{h1}} \times \frac{\partial a_{h1}}{\partial S_{h1}} \times \frac{\partial S_{h1}}{\partial w_1}$$

However this time around, we are a lot further away from the output neurons — hence, to find the values of the individual components of the RHS of this equation, there is going to be a lot more “chaining”…

For the first derivative:

Zoom image will be displayed

$$\frac{\partial E_{total}}{\partial a_{h1}} = \frac{\partial E_{01}}{\partial a_{h1}} + \frac{\partial E_{02}}{\partial a_{h1}}$$

Where:

Zoom image will be displayed

$$\frac{\partial E_{01}}{\partial a_{h1}} = \frac{\partial E_{01}}{\partial a_{01}} \times \frac{\partial a_{01}}{\partial S_{01}} \times \frac{\partial S_{01}}{\partial a_{h1}}$$

Zoom image will be displayed

$$\frac{\partial E_{02}}{\partial a_{h1}} = \frac{\partial E_{02}}{\partial a_{02}} \times \frac{\partial a_{02}}{\partial S_{02}} \times \frac{\partial S_{02}}{\partial a_{h1}}$$

Now, since we have calcuated δ_{o1} and δ_{o2} previously (see the calculations made in the output layer section of this article), we can substitute in the values of these deltas into the equation.

Zoom image will be displayed

$$\frac{\partial E_{01}}{\partial a_{h1}} = \delta_{01} \times \frac{\partial S_{01}}{\partial a_{h1}}$$

Zoom image will be displayed

$$\frac{\partial E_{02}}{\partial a_{h1}} = \delta_{02} \times \frac{\partial S_{02}}{\partial a_{h1}}$$

Hence, the derivative of the weighted sum with respect to the previous layer's neurons is essentially just the corresponding weight.

Zoom image will be displayed

$$\frac{\partial S_{01}}{\partial a_{h1}} = \frac{\partial (w_7 \times a_{h1} + w_9 \times a_{h2} + w_{11} \times a_{h3})}{\partial a_{h1}} = w_7$$

Zoom image will be displayed

$$\frac{\partial S_{02}}{\partial a_{h1}} = \frac{\partial (w_8 \times a_{h1} + w_10 \times a_{h2} + w_{12} \times a_{h3})}{\partial a_{h1}} = w_8$$

Now, substituting these values in for the partial error term:

Zoom image will be displayed

$$\frac{\partial E_{\text{total}}}{\partial a_{h1}} = \frac{\partial E_{01}}{\partial a_{h1}} + \frac{\partial E_{02}}{\partial a_{h1}} = \boxed{\delta_{01} \times w_7 + \delta_{02} \times w_8}$$

Zoom image will be displayed

$$\frac{\partial E_{\text{total}}}{\partial a_{h1}} = \frac{\partial E_{01}}{\partial a_{h1}} + \frac{\partial E_{02}}{\partial a_{h1}}$$
$$= \boxed{\delta_{01} \times w_7 + \delta_{02} \times w_8}$$

$$\therefore \frac{\partial E_{\text{total}}}{\partial w_1} = (\delta_{01} \times w_7 + \delta_{02} \times w_8) \times \frac{\partial a_{h1}}{\partial s_{h1}} \times \frac{\partial s_{h1}}{\partial w_1}$$

The value of $\partial a_{h1} / \partial s_{h1}$ would just be the derivative of the sigmoid function

Zoom image will be displayed

$$\frac{\partial a_{h1}}{\partial s_{h1}} = a_{h1}(1 - a_{h1})$$

And the value of $\partial s_{h1} / \partial w_1$ is the output of the previous layer neuron (which in this case, is the input layer neuron since there is only one hidden layer)

Zoom image will be displayed

$$\frac{\partial s_{h1}}{\partial w_1} = i_1$$

So putting it all together:

Zoom image will be displayed

$$\frac{\partial E_{\text{total}}}{\partial w_1} = (\delta_{01} \times w_7 + \delta_{02} \times w_8) \times a_{h1} \times (1 - a_{h1}) \times i_1$$

I hope you can see what has occurred in these steps — a similar working process can be done to find the formula of all of the weights (which I won't show).

But in essence, to find the value of an updated weight, first calculate delta of the weight's output neuron, and then subtract the old weight from the delta, multiplied by the delta, multiplied by the previous value of the weight's input neuron.

If that is difficult to understand, then the calculations below may help you see what is occurring numerically.

The Hidden Layer (Calculation)

Previous delta values calculated:

δ_{o1} = -0.0984

δ_{o2} = 0.1479

Hidden layer 1:

Zoom image will be displayed

$$\delta_{h1} = (w_7 \times \delta_{01} + w_8 \times \delta_{02})[a_{h1} \times (1 - a_{h1})]$$
$$= (0.25 \times -0.0984 + 0.5 \times 0.1479)[0.5461(1 - 0.5461)]$$
$$= 0.0122$$

Zoom image will be displayed

$$w_1' = 0.1 - 0.3 \times 0.0122 \times 0.85$$
$$= 0.0969$$

Zoom image will be displayed

$$w_4' = 0.4 - 0.3 \times 0.0122 \times 0.25$$
$$= 0.3991$$

Hidden layer 2:

Zoom image will be displayed

$$\delta_{h2} = (w_9 \times \delta_{01} + w_{10} \times \delta_{02})[a_{h2} \times (1 - a_{h2})]$$
$$= (0.1 \times -0.0984 + 0.2 \times 0.1479)[0.5732(1 - 0.5732)]$$
$$= 0.0048$$

Zoom image will be displayed

$$w_2' = 0.2 - 0.3 \times 0.0048 \times 0.85$$
$$= 0.1988$$

Zoom image will be displayed

$$w_5' = 0.5 - 0.3 \times 0.0048 \times 0.25$$
$$= 0.4996$$

Hidden layer 3:

Zoom image will be displayed

$$\delta_{h3} = (w_{11} \times \delta_{01} + w_{12} \times \delta_{02})[a_{h3} \times (1 - a_{h3})]$$
$$= (0.3 \times -0.0984 + 0.4 \times 0.1479)[0.5998(1 - 0.5998)]$$
$$= 0.0071$$

Zoom image will be displayed

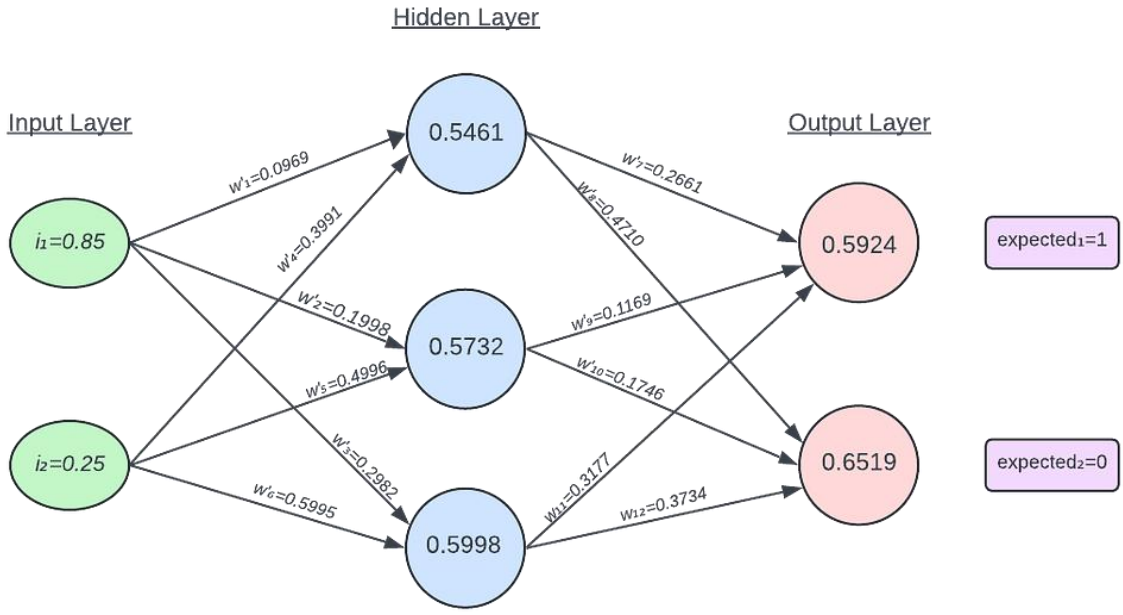
$$w_3' = 0.3 - 0.3 \times 0.0071 \times 0.85$$
$$= 0.2982$$

Zoom image will be displayed

$$w_6' = 0.6 - 0.3 \times 0.0071 \times 0.25$$
$$= 0.5995$$

And we're done!

Zoom image will be displayed



Neural network with updated weights

Closing Thoughts

The following was a complete example of a forward and back propagation for a neural network with 3 layers.

Typically neural networks are trained on multiple instances of data and can also be trained for multiple iterations (we call these **epochs)**. Doing this will gradually increase/decrease the weights depending on the instance, until a neural network is optimised for a set of instances.

This process was *very* laborious and math heavy — thankfully that is why we have computers simulate all of this work. Libraries like PyTorch abstract many of the mathematical complexities and should definitely be used for any sort of model training.

Nonetheless, a complete walkthrough of the maths would definitely help reinforce the understanding needed when implementing this model.